

8교시: Delivery handoff와 구름 EXP 배움일기



수업 목표

- Day 3의 image 납품 증거를 README/runbook 형태로 정리한다.
- build/run/verify/scan/cleanup과 failure RCA를 한 페이지에 남긴다.
- Day 4 runtime config/observability로 넘길 image 기준을 확정한다.

개념 설명

운영 인수인계는 명령어 목록만 넘기는 일이 아니다. 다음 사람이 어느 directory에서 build하는지, image tag가 무엇인지, 어떤 port로 실행하는지, 정상 확인은 무엇인지, 취약점 scan 결과는 어떤지, 실패하면 어디부터 볼지 알아야 한다.

이미 week2/day3/labs/static-site/README.md가 lab runbook 역할을 한다. 8교시는 이 README를 학생이 직접 읽고, 실행 결과값을 자기 handoff note에 옮기는 시간이다. Day 4에서 runtime config, logs, inspect, exec, stats를 다루려면 Day 3 image가 무엇이고 어떻게 검증됐고 어떤 보안 scan 상태인지 명확해야 한다.

인수인계 표

학생은 Day 3 결과를 문장형 감상문이 아니라 표로 정리한다. 표로 정리하면 다음 사람이 build 명령, 실행 기준, 보안 점검, 실패 복구 기준을 빠르게 확인할 수 있다.

구분	작성할 내용	확인 명령 또는 증거	합격 기준
소스 위치	실습 앱 directory	<code>pwd, ls -la</code>	week2/day3/labs/static-site와 필수 파일 확인
이미지 태그	최종 제출 image tag	<code>docker images</code> <code>paperclip-static-site</code>	day3, day3-reviewed, 버전/환경 tag 의 미 설명 가능
빌드 명령	image를 만든 명령	<code>docker build -t ...</code> .	build가 성공하고 image 목록에 표시됨
실행 명령	container 실행 명령	<code>docker run -d --name ... -p ...</code>	container 이름과 port mapping 설명 가능
정상 확인	HTTP 응답 기준	<code>curl -I, curl -s</code>	HTTP/1.1 200 OK와 페이지 문구 확인
취약점 점검	Docker Scout 결과 또는 blocker	<code>docker scout cves ...</code>	critical/high 없음, 조치 필요, 실행 불가 사유 중 하나 기록
이미지 증거	layer와 metadata	<code>docker history,</code> <code>docker image inspect</code>	COPY index.html, COPY styles.css, image id/size/tag 확인
실패 분석	재현한 실패와 원인	실패 출력, <code>docker logs,</code> <code>docker ps, find</code>	build/run/verify/hygiene 중 어느 단계 문제인지 분류
복구 확인	수정 후 다시 확인한 결과	재실행한 build/run/curl 명령	같은 문제가 해결됐음을 출력으로 확인
정리 작업	container/image/lab 복사본 정리	cleanup 명령	불필요한 container와 임시 directory 제거
registry 판단	push 여부	push gate 체크	local only 또는 push candidate 사유 설명
secret/context 점검	민감 정보 포함 여부	<code>.dockerignore, find,</code> <code>du -sh</code>	<code>.env, token, dependency/cache</code> 제외 기준 확인

작성 예시는 다음처럼 짧고 명확하게 남긴다.

구분	기록 예시
이미지 태그	<code>paperclip-static-site:day3-reviewed</code> , 수업 검수 완료 tag
정상 확인	<code>curl -I http://localhost:18083</code> 결과 HTTP/1.1 200 OK
취약점 점검	Scout 실행 가능, critical/high 없음 또는 scan blocker 기록
실패 분석	missing file 실패, <code>COPY index.html ... not found</code> , build context 문제
registry 판단	secret/context 확인 전이므로 push하지 않고 local only

최종 확인 명령과 기대 결과

```
docker images paperclip-static-site
docker ps -a --filter name=paperclip-day3-static
docker history paperclip-static-site:day3
docker image inspect paperclip-static-site:day3 --format "{{.Id}} {{.Size}}
{{json .RepoTags}}"
docker scout cves --only-severity critical,high paperclip-static-site:day3 ||
true
```

Expected pattern:

```
paperclip-static-site    day3
paperclip-static-site    day3-v2
paperclip-static-site    day3-reviewed
paperclip-day3-static    Up 또는 Exited
COPY index.html ./index.html
COPY styles.css ./styles.css
sha256:<id> <size> ["paperclip-static-site:day3"]
critical/high CVE 없음 또는 scan blocker/action note 기록
```

Cleanup

```
docker stop paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-
bad-cmd || true
docker rm paperclip-day3-static paperclip-day3-static-wrong paperclip-day3-
bad-cmd || true
rm -rf week2/day3/labs/static-site-broken
# 필요할 때만 image 삭제
# docker image rm paperclip-static-site:day3 paperclip-static-site:day3-v2
paperclip-static-site:day3-reviewed paperclip-static-site:v1.0.0 paperclip-
static-site:staging paperclip-static-site:size-default paperclip-static-
site:size-alpine paperclip-static-site:size-trixie paperclip-static-
site:broken paperclip-static-site:broken-fixed paperclip-static-site:bad-cmd
```

구름 EXP 배움일기

- 내가 만든 image의 tag와 acceptance check
- Dockerfile instruction 중 실제 운영 계약으로 보인 것
- build context와 .dockerignore에서 막아야 할 위험

- build/run/verify/scan 중 내가 확인한 blocker 또는 RCA
- Day 4에서 같은 image를 다른 runtime config로 실행할 때 확인하고 싶은 질문

핵심 포인트

Day 3의 완료 기준은 이미지 하나 만들었다가 아니라 다른 사람이 build/run/verify/scan/failure recovery를 재현할 수 있다.